GH-600 Hands-on Workshop

CityOps AI — Complete Step-by-Step Workbook

Build a small, governed AI-assisted municipal incident workflow using Python, GitHub, Copilot, MCP, and the OpenAI API.

From an empty folder • All 8 stages • Complete file contents • PowerShell-first

Design principles

- Minimal command-line Python application; no web server, Docker, database, or agent framework.
- Dependencies are limited to the OpenAI SDK and MCP SDK; standard library for the rest.
- OpenAI Developer Platform only — this workbook does not use Azure OpenAI.
- Deterministic validation and human approval remain authoritative; the demo never dispatches real municipal work.
- Each stage is a complete checkpoint. Save each file exactly as shown and replace earlier versions when instructed.

Version: October 2026

How to use this workbook

Follow the stages in order. The repository evolves through eight checkpoints. When a stage says “replace”, overwrite the entire file with the supplied content; do not append fragments. The workshop is designed for Windows, PowerShell, VS Code, and GitHub.

Workshop outcomes

• Create a small incident-triage application and add tests.

• Add planning, a bounded OpenAI suggestion step, tools, MCP, state, and an audit trail.

• Use a supervisor to coordinate triage, routing, and safety-review roles.

• Evaluate behaviour with repeatable cases and explicit acceptance criteria.

• Use GitHub Issues, branches, pull requests, Actions, Copilot instructions/prompts/agents/skills, and security automation.

• Finish with a documented, tested, auditable workflow.

Prerequisites

• Python 3.11 or newer installed and available as `python` in PowerShell.

• Git installed; a GitHub account and a repository you can administer.

• VS Code with GitHub Copilot access for the Copilot exercises.

• Optional: OpenAI API key and access to an OpenAI model. Stage 1–3 and deterministic fallbacks work without it.

• GitHub CLI (`gh`) is optional; browser-based GitHub steps are provided.

Safety boundary

CityOps AI is a teaching simulation. It classifies and proposes a team; it does not contact emergency services, create real work orders, or dispatch crews. Never use real personal data. The approval function in this sample is not production identity verification.

Eight-stage map

Stage	Focus	Checkpoint
1	Minimal baseline + tests + Copilot repo guidance	Classify a sample request
2	Planning, bounded OpenAI suggestion, validation	Model suggestion cannot override policy
3	Local tools + MCP server	Call mission/status tools

Stage	Focus	Checkpoint
4	Persistent state, JSONL audit, recovery	Create, inspect, resume a request
5	Supervisor + triage/routing/safety agents	Produce a reviewed plan
6	Evaluation cases, tests, specifications	Run evaluation and unit tests
7	GitHub CI/CD + security automation	Pull request runs checks
8	Copilot workflows + governance + capstone	Complete PR and release checklist

Stage 1 — Create the baseline repository

What you learn

Start with a deterministic classifier, a tiny command-line entry point, unit tests, and repository instructions. Keep the first version boring and predictable; later stages add AI and orchestration without removing the policy checks.

1.1 Create the folder and virtual environment

```
mkdir cityops-ai
cd cityops-ai
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
mkdir app, tests, .github
New-Item app\__init__.py -ItemType File
New-Item tests\__init__.py -ItemType File
```

1.2 Create `app.py` — complete file

```
"""CityOps AI: deterministic municipal request triage demo."""
from dataclasses import dataclass
from typing import Optional


@dataclass
class Classification:
    service_type: str
    priority: str
    assigned_team: str
    rationale: str


RULES = {
    "streetlight": ("Electrical Maintenance", "normal",
                    "Streetlight issue matched the electrical-maintenance rule."),
    "water_leak": ("Water Services", "high",
                   "Water leak matched the water-services rule."),
    "road_damage": ("Roads Maintenance", "normal",
                    "Road damage matched the roads-maintenance rule."),
}


def classify_request(description: str) -> Classification:
    """Return a deterministic classification; unknown cases are escalated."""
    text = description.strip().lower()
    if not text:
        return Classification("unknown", "review", "Human Review",
                              "Description is empty; human review is required.")
    if any(word in text for word in ("burst pipe", "major leak", "flooding")):
        key = "water_leak"
    elif any(word in text for word in ("pothole", "road damage", "broken road")):
        key = "road_damage"
    elif any(word in text for word in ("streetlight", "lamp post", "dark street")):
        key = "streetlight"
    elif any(word in text for word in ("water leak", "leaking water", "leak")):
        key = "water_leak"
```

```
    else:
        return Classification("unknown", "review", "Human Review",
                              "No rule matched; human review is required.")
    team, priority, rationale = RULES[key]
    return Classification(key, priority, team, rationale)


def main() -> None:
    print("CityOps AI — demo only")
    description = input("Describe the municipal issue: ").strip()
    result = classify_request(description)
    print(f"Service type: {result.service_type}")
    print(f"Priority: {result.priority}")
    print(f"Assigned team: {result.assigned_team}")
    print(f"Rationale: {result.rationale}")


if __name__ == "__main__":
    main()
```

1.3 Create `test_app.py` — complete file

```
import unittest
from app import classify_request


class ClassificationTests(unittest.TestCase):
    def test_streetlight(self):
        result = classify_request("Streetlight is not working")
        self.assertEqual(result.service_type, "streetlight")
        self.assertEqual(result.assigned_team, "Electrical Maintenance")

    def test_water_leak_is_high_priority(self):
        result = classify_request("There is a major leak")
        self.assertEqual(result.service_type, "water_leak")
        self.assertEqual(result.priority, "high")

    def test_unknown_goes_to_human_review(self):
        result = classify_request("Please inspect the park")
        self.assertEqual(result.service_type, "unknown")
        self.assertEqual(result.assigned_team, "Human Review")


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

1.4 Create `.gitignore`

```
# Python
.venv/
__pycache__/
*.py[cod]
.coverage
# Local secrets
.env
# Generated demo state and audit records
cityops_state.json
cityops_audit.jsonl
# Editor
.vscode/
```

1.5 Create `.github/copilot-instructions.md`

```
# CityOps AI repository guidance

- This is a small Python command-line teaching demo, not a production dispatch system.
- Prefer the Python standard library; keep dependencies minimal.
- Never send real citizen data to a model. Use synthetic examples only.
- Deterministic validation and human review are authoritative.
- Unknown or conflicting requests must be escalated, not guessed.
- Do not perform real-world dispatch or call external municipal systems.
- Add or update unittest tests for behaviour changes.
- When changing a file for a workshop stage, provide a complete replacement file.
```

1.6 Run and verify

```
python app.py
# Try: "Streetlight is not working"
python -m unittest -v
```

Expected: a streetlight request routes to Electrical Maintenance; an unknown request routes to Human Review. All three tests pass.

1.7 Git checkpoint

```
git init
git add .
git commit -m "stage 1: add deterministic CityOps baseline"
```

GitHub exercise: create a repository named `cityops-ai`, add it as `origin`, and push the branch. If using GitHub CLI:

```
gh repo create cityops-ai --private --source . --remote origin --push
```

Stage 2 — Add planning and an optional OpenAI suggestion

Concept

The model can suggest a classification, but the deterministic classifier and validation rules remain authoritative. A missing key, API error, or malformed response must not stop the demo.

2.1 Install the only model dependency and create the environment file

```
python -m pip install openai
@'
OPENAI_API_KEY=
OPENAI_MODEL=gpt-4.1-mini
'@ | Set-Content .env.example
@'
openai
mcp
'@ | Set-Content requirements.txt
```

2.2 Create `openai_client.py` — complete file

```
"""Optional OpenAI suggestion; safe to skip when no key is configured."""
import json
import os
from pathlib import Path

try:
    from openai import OpenAI
except ImportError:
    OpenAI = None


def _load_env_file() -> None:
    """Tiny .env reader to avoid an additional dependency."""
    path = Path(".env")
    if not path.exists():
        return
    for raw in path.read_text(encoding="utf-8").splitlines():
        line = raw.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))


def suggest_classification(description: str):
    """Return a model suggestion dict, or None if unavailable."""
    _load_env_file()
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key or OpenAI is None:
        return None
    try:
        client = OpenAI(api_key=api_key)
        response = client.responses.create(
            model=os.getenv("OPENAI_MODEL", "gpt-4.1-mini"),
            input=(
                "Classify this synthetic municipal request. Return only JSON with "
                "keys service_type, priority, assigned_team, rationale. "
                "Allowed service_type values: streetlight, water_leak, road_damage, unknown. "
```

```
                "Allowed priority values: low, normal, high, review. "
                "Unknown or ambiguous requests must use service_type=unknown, "
                "priority=review, assigned_team=Human Review. Request: "
                + description[:1000]
            ),
        )
        text = response.output_text.strip()
        result = json.loads(text)
        if not isinstance(result, dict):
            return None
        return result
    except Exception:
        # Workshop fallback: API errors must not break deterministic processing.
        return None
```

2.3 Replace `app.py` — complete file

```
"""CityOps AI: deterministic rules plus an optional model suggestion."""
from dataclasses import dataclass
from openai_client import suggest_classification


@dataclass
class Classification:
    service_type: str
    priority: str
    assigned_team: str
    rationale: str


RULES = {
    "streetlight": ("Electrical Maintenance", "normal"),
    "water_leak": ("Water Services", "high"),
    "road_damage": ("Roads Maintenance", "normal"),
}


def classify_request(description: str) -> Classification:
    text = description.strip().lower()
    if not text:
        return Classification("unknown", "review", "Human Review",
                              "Empty description; human review required.")
    if any(w in text for w in ("burst pipe", "major leak", "flooding",
                               "water leak", "leaking water", " leak")):
        key = "water_leak"
    elif any(w in text for w in ("pothole", "road damage", "broken road")):
        key = "road_damage"
    elif any(w in text for w in ("streetlight", "lamp post", "dark street")):
        key = "streetlight"
    else:
        return Classification("unknown", "review", "Human Review",
                              "No rule matched; human review required.")
    team, priority = RULES[key]
    return Classification(key, priority, team,
                          f"Deterministic rule matched: {key}.")


def validate_classification(result: Classification) -> Classification:
    """Fail closed: unsupported values and unknown types go to human review."""
    if result.service_type not in RULES:
        return Classification("unknown", "review", "Human Review",
                              "Validation rejected unknown service type.")
```

```
    expected_team, expected_priority = RULES[result.service_type]
    if result.assigned_team != expected_team or result.priority != expected_priority:
        return Classification("unknown", "review", "Human Review",
                              "Validation rejected conflicting assignment.")
    return result


def get_assessment(description: str) -> dict:
    deterministic = classify_request(description)
    suggestion = suggest_classification(description)
    chosen = validate_classification(deterministic)
    conflict = False
    if suggestion:
        expected = {
            "service_type": deterministic.service_type,
            "priority": deterministic.priority,
            "assigned_team": deterministic.assigned_team,
        }
        conflict = any(suggestion.get(k) != v for k, v in expected.items())
    return {
        "description": description,
        "classification": chosen.__dict__,
        "model_suggestion": suggestion,
        "model_conflict": conflict,
        "requires_human_review": (
            chosen.service_type == "unknown" or conflict
        ),
    }


def main() -> None:
    description = input("Describe the municipal issue: ").strip()
    result = get_assessment(description)
    print("Assessment:")
    for key, value in result.items():
        print(f"{key}: {value}")


if __name__ == "__main__":
    main()
```

2.4 Configure the API key safely

• Copy `.env.example` to `.env` and add your OpenAI API key locally.

• Never put the key in Python source, README, screenshots, GitHub Issues, or commits.

• `.env` is ignored by Git. Do not paste a real key into workshop chat.

• The model call is optional; leave the key blank to test the deterministic fallback.

```
Copy-Item .env.example .env
python -m pip install -r requirements.txt
python -m unittest -v
python app.py
```

2.5 Checkpoint

Test one known request and one ambiguous request. Confirm an unknown request is sent to Human Review. Discuss why model output must be validated rather than trusted.

Stage 3 — Add tools and a local MCP server

Concept

Tools are small, explicit functions. MCP exposes a tool interface to compatible clients; it does not automatically make the application safe. Keep tool inputs bounded and outputs synthetic.

3.1 Create `tools.py` — complete file

```
"""Small deterministic tools used by CityOps AI."""
from datetime import datetime, timezone


def get_service_catalog() -> dict:
    return {
        "streetlight": {"team": "Electrical Maintenance", "priority": "normal"},
        "water_leak": {"team": "Water Services", "priority": "high"},
        "road_damage": {"team": "Roads Maintenance", "priority": "normal"},
    }


def get_demo_status(request_id: str) -> dict:
    """Return a synthetic status; this does not query a real system."""
    safe_id = "".join(ch for ch in request_id if ch.isalnum() or ch in "-_")[:40]
    if not safe_id:
        return {"status": "invalid", "request_id": "", "message": "A request ID is required."}
    return {
        "request_id": safe_id,
        "status": "simulated",
        "checked_at_utc": datetime.now(timezone.utc).isoformat(),
        "message": "Demo-only status. No real service was contacted.",
    }


def make_work_order_draft(service_type: str, description: str) -> dict:
    catalog = get_service_catalog()
    if service_type not in catalog:
        return {"created": False, "reason": "Unknown type; human review required."}
    return {
        "created": False,
        "draft_only": True,
        "service_type": service_type,
        "team": catalog[service_type]["team"],
        "description": description.strip()[:500],
        "message": "Draft only; no work order was submitted.",
    }
```

3.2 Create `mcp_server.py` — complete file

```
"""Local stdio MCP server exposing synthetic CityOps tools."""
from mcp.server.fastmcp import FastMCP
from tools import get_demo_status, get_service_catalog, make_work_order_draft

mcp = FastMCP("cityops-tools")


@mcp.tool()
```

```
def service_catalog() -> dict:
    """List the allowed demo service types and assigned teams."""
    return get_service_catalog()


@mcp.tool()
def demo_request_status(request_id: str) -> dict:
    """Return synthetic status for a request ID; never calls a real system."""
    return get_demo_status(request_id)


@mcp.tool()
def draft_work_order(service_type: str, description: str) -> dict:
    """Prepare a local draft only; never submit a real work order."""
    return make_work_order_draft(service_type, description)


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

3.3 Install MCP and test local tools

```
python -m pip install -r requirements.txt
python -c "from tools import get_service_catalog, make_work_order_draft; print(get_service_catalog()); print(make_work_order_draft('streetlight','Lamp is out'))"
python mcp_server.py
```

The last command waits for an MCP client because stdio is a client/server protocol, not a normal interactive prompt. Stop it with Ctrl+C after confirming it starts.

3.4 Configure a compatible local MCP client

MCP client configuration varies by product and version. Use this logical definition in the configuration UI/file for a client that supports local stdio servers. Paths may need to be absolute.

```
{
  "mcpServers": {
    "cityops-tools": {
      "command": "python",
      "args": ["mcp_server.py"]
    }
  }
}
```

3.5 Copilot exercise

• Ask Copilot to explain `@mcp.tool()` and trace a call from the client to the function.

• Ask it to add a tool for listing allowed service types, then inspect the complete file and test it.

• Reject suggestions that submit work orders, call external services, or bypass validation.

Checkpoint

You can run each tool directly from Python and can start the MCP server. A compatible client can discover the three tools. No real external system is contacted.

Stage 4 — Add persistent state, audit trail, and recovery

Concept

Persist demo requests as JSON and append audit events as JSON Lines. Treat the audit trail as a useful local record, not tamper-proof compliance storage.

4.1 Create `state.py` — complete file

```
"""Local JSON state and append-only JSONL audit for the demo."""
import json
from datetime import datetime, timezone
from pathlib import Path

STATE_FILE = Path("cityops_state.json")
AUDIT_FILE = Path("cityops_audit.jsonl")


def utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()


def load_state() -> dict:
    if not STATE_FILE.exists():
        return {"requests": {}}
    try:
        data = json.loads(STATE_FILE.read_text(encoding="utf-8"))
        if isinstance(data, dict) and isinstance(data.get("requests"), dict):
            return data
    except (OSError, json.JSONDecodeError):
        pass
    return {"requests": {}}


def save_state(state: dict) -> None:
    temp = STATE_FILE.with_suffix(".tmp")
    temp.write_text(json.dumps(state, indent=2), encoding="utf-8")
    temp.replace(STATE_FILE)


def audit(event: str, request_id: str, details: dict | None = None) -> None:
    record = {
        "timestamp_utc": utc_now(),
        "event": event,
        "request_id": request_id,
        "details": details or {},
    }
    with AUDIT_FILE.open("a", encoding="utf-8") as handle:
        handle.write(json.dumps(record, ensure_ascii=False) + "\n")


def create_request(request_id: str, description: str, assessment: dict) -> dict:
    state = load_state()
    if request_id in state["requests"]:
        raise ValueError("Request ID already exists.")
    item = {
        "request_id": request_id,
        "description": description[:500],
        "assessment": assessment,
```

```
        "status": "awaiting_review" if assessment["requires_human_review"] else "ready_for_approval",
        "created_at_utc": utc_now(),
        "approved_by": None,
    }
    state["requests"][request_id] = item
    save_state(state)
    audit("request_created", request_id, {"status": item["status"]})
    return item


def get_request(request_id: str) -> dict | None:
    return load_state()["requests"].get(request_id)


def approve_request(request_id: str, approver: str) -> dict:
    state = load_state()
    item = state["requests"].get(request_id)
    if not item:
        raise KeyError("Request not found.")
    if item["assessment"]["requires_human_review"]:
        raise ValueError("Human-review cases cannot be approved by this demo shortcut.")
    if not approver.strip():
        raise ValueError("Approver label is required.")
    item["status"] = "approved_demo_only"
    item["approved_by"] = approver.strip()[:80]
    item["approved_at_utc"] = utc_now()
    save_state(state)
    audit("request_approved_demo_only", request_id, {"approver": item["approved_by"]})
    return item
```

4.2 Replace `app.py` — complete file

```
"""CityOps AI CLI with optional model suggestion and local state."""
import argparse
import json
from app import get_assessment
from state import create_request, get_request, approve_request


def main():
    parser = argparse.ArgumentParser(description="CityOps AI demo CLI")
    sub = parser.add_subparsers(dest="command", required=True)

    create = sub.add_parser("create", help="Create a synthetic request")
    create.add_argument("--id", required=True)
    create.add_argument("--description", required=True)

    show = sub.add_parser("show", help="Show a saved request")
    show.add_argument("--id", required=True)

    approve = sub.add_parser("approve", help="Mark a safe request approved in demo state")
    approve.add_argument("--id", required=True)
    approve.add_argument("--approver", required=True)

    args = parser.parse_args()
    try:
        if args.command == "create":
            item = create_request(args.id, args.description, get_assessment(args.description))
        elif args.command == "show":
            item = get_request(args.id)
            if item is None:
                raise KeyError("Request not found.")
```

```
        else:
            item = approve_request(args.id, args.approver)
        print(json.dumps(item, indent=2))
    except (ValueError, KeyError) as exc:
        parser.error(str(exc))


if __name__ == "__main__":
    main()
```

4.3 Run the lifecycle

```
python app.py create --id DEMO-001 --description "Streetlight is not working"
python app.py show --id DEMO-001
python app.py approve --id DEMO-001 --approver "Workshop Facilitator"
Get-Content .\cityops_audit.jsonl
```

Try an unknown request and confirm the approval shortcut refuses it. The JSON and JSONL files are local generated artifacts and are ignored by Git.

Checkpoint

You can create, show, and approve a safe demo request; the state persists between runs; audit events are appended. Do not describe the local approver label as real authentication.

Stage 5 — Coordinate multiple roles with a supervisor

Concept

A supervisor runs a bounded workflow: triage → routing → safety review → final policy checks. These are simple Python functions representing agent roles, not autonomous agents with unrestricted tools.

5.1 Replace `app.py` — complete file

```
"""CityOps AI: supervisor orchestration, state, audit, and safety checks."""
import argparse
import json
from app import get_assessment
from state import create_request, get_request, approve_request, audit


def triage_agent(description: str) -> dict:
    assessment = get_assessment(description)
    return {
        "role": "triage",
        "service_type": assessment["classification"]["service_type"],
        "priority": assessment["classification"]["priority"],
        "rationale": assessment["classification"]["rationale"],
        "requires_human_review": assessment["requires_human_review"],
        "model_conflict": assessment["model_conflict"],
    }


def routing_agent(triage: dict) -> dict:
    routes = {
        "streetlight": "Electrical Maintenance",
        "water_leak": "Water Services",
        "road_damage": "Roads Maintenance",
    }
    service_type = triage["service_type"]
    return {
        "role": "routing",
        "assigned_team": routes.get(service_type, "Human Review"),
        "route_is_allowed": service_type in routes,
    }


def safety_reviewer(triage: dict, routing: dict) -> dict:
    issues = []
    if triage["requires_human_review"]:
        issues.append("Triage requires human review.")
    if not routing["route_is_allowed"]:
        issues.append("No approved route exists for this service type.")
    if triage["model_conflict"]:
        issues.append("Model suggestion conflicts with deterministic rules.")
    return {
        "role": "safety-reviewer",
        "passed": not issues,
        "issues": issues,
        "human_approval_required": True,
    }

def supervisor_plan(description: str) -> dict:
```

```
    triage = triage_agent(description)
    routing = routing_agent(triage)
    review = safety_reviewer(triage, routing)
    allowed = review["passed"] and routing["route_is_allowed"]
    return {
        "description": description[:500],
        "agents": [triage, routing, review],
        "final_plan": {
            "service_type": triage["service_type"],
            "priority": triage["priority"],
            "assigned_team": routing["assigned_team"],
            "status": "ready_for_human_approval" if allowed else "escalate_to_human",
            "can_auto_dispatch": False,
        },
        "requires_human_review": not allowed,
    }


def main():
    parser = argparse.ArgumentParser(description="CityOps AI supervised demo")
    sub = parser.add_subparsers(dest="command", required=True)
    create = sub.add_parser("create")
    create.add_argument("--id", required=True)
    create.add_argument("--description", required=True)
    show = sub.add_parser("show")
    show.add_argument("--id", required=True)
    approve = sub.add_parser("approve")
    approve.add_argument("--id", required=True)
    approve.add_argument("--approver", required=True)
    plan = sub.add_parser("plan")
    plan.add_argument("--description", required=True)
    args = parser.parse_args()

    try:
        if args.command == "plan":
            result = supervisor_plan(args.description)
            print(json.dumps(result, indent=2))
            return
        if args.command == "create":
            result = supervisor_plan(args.description)
            assessment = {
                "classification": result["agents"][0],
                "model_suggestion": None,
                "model_conflict": result["agents"][0]["model_conflict"],
                "requires_human_review": result["requires_human_review"],
                "plan": result,
            }
            item = create_request(args.id, args.description, assessment)
            audit("supervisor_plan_created", args.id, result["final_plan"])
        elif args.command == "show":
            item = get_request(args.id)
            if item is None:
                raise KeyError("Request not found.")
        else:
            item = approve_request(args.id, args.approver)
        print(json.dumps(item, indent=2))
    except (ValueError, KeyError) as exc:
        parser.error(str(exc))


if __name__ == "__main__":
    main()
```

5.2 Try the supervisor

```
python app.py plan --description "Streetlight is not working"
python app.py plan --description "Please inspect the park"
python app.py create --id DEMO-002 --description "Streetlight is not working"
python app.py show --id DEMO-002
```

Inspect the three roles in the output. The safety reviewer can block a plan; all plans still require a human decision. `can_auto_dispatch` is always false.

5.3 GitHub Copilot exercise

• Ask Copilot to explain the data passed between the three roles.

• Ask it to propose a unit test for an unknown service type, but require it to preserve the human-review rule.

• Use a feature branch and pull request for any change; do not accept a suggestion without running tests.

Stage 6 — Add evaluation, acceptance criteria, and tests

Concept

Evaluation uses synthetic cases and deterministic assertions. It tests the behaviour of the system, not just whether a model produces fluent text.

6.1 Create `cases.json`

```
[
  {
    "id": "streetlight-01",
    "description": "Streetlight is not working",
    "expected_type": "streetlight",
    "expected_team": "Electrical Maintenance",
    "must_review": false
  },
  {
    "id": "water-01",
    "description": "A major water leak is flooding the pavement",
    "expected_type": "water_leak",
    "expected_team": "Water Services",
    "must_review": false
  },
  {
    "id": "road-01",
    "description": "Large pothole on the road",
    "expected_type": "road_damage",
    "expected_team": "Roads Maintenance",
    "must_review": false
  },
  {
    "id": "unknown-01",
    "description": "Please look into the park",
    "expected_type": "unknown",
    "expected_team": "Human Review",
    "must_review": true
  },
  {
    "id": "empty-01",
    "description": "",
    "expected_type": "unknown",
    "expected_team": "Human Review",
    "must_review": true
  }
]
```

6.2 Create `evaluate.py`

```
"""Run a repeatable evaluation over synthetic CityOps examples."""
import json
from pathlib import Path
from app import supervisor_plan


def run_evaluation(path: str = "cases.json") -> tuple[int, int]:
    cases = json.loads(Path(path).read_text(encoding="utf-8"))
    passed = 0
    for case in cases:
```

```
        plan = supervisor_plan(case["description"])
        final = plan["final_plan"]
        checks = {
            "type": final["service_type"] == case["expected_type"],
            "team": final["assigned_team"] == case["expected_team"],
            "review": plan["requires_human_review"] == case["must_review"],
            "no_dispatch": final["can_auto_dispatch"] is False,
        }
        ok = all(checks.values())
        print(f'{"PASS" if ok else "FAIL"} {case["id"]}: {checks}')
        passed += int(ok)
    print(f"Evaluation: {passed}/{len(cases)} passed")
    return passed, len(cases)


if __name__ == "__main__":
    passed, total = run_evaluation()
    raise SystemExit(0 if passed == total else 1)
```

6.3 Replace `test_app.py`

```
import unittest
from app import classify_request, validate_classification, supervisor_plan, Classification


class CityOpsTests(unittest.TestCase):
    def test_streetlight(self):
        result = classify_request("Streetlight is not working")
        self.assertEqual(result.service_type, "streetlight")
        self.assertEqual(result.assigned_team, "Electrical Maintenance")

    def test_water_leak(self):
        result = classify_request("Major water leak")
        self.assertEqual(result.service_type, "water_leak")
        self.assertEqual(result.priority, "high")

    def test_unknown_escalates(self):
        result = classify_request("Inspect the park")
        self.assertEqual(result.service_type, "unknown")
        self.assertEqual(result.assigned_team, "Human Review")

    def test_invalid_assignment_fails_closed(self):
        bad = Classification("streetlight", "high", "Water Services", "bad")
        checked = validate_classification(bad)
        self.assertEqual(checked.service_type, "unknown")
        self.assertEqual(checked.assigned_team, "Human Review")

    def test_supervisor_never_dispatches(self):
        result = supervisor_plan("Streetlight is not working")
        self.assertFalse(result["final_plan"]["can_auto_dispatch"])

    def test_unknown_requires_review(self):
        result = supervisor_plan("Inspect the park")
        self.assertTrue(result["requires_human_review"])


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

6.4 Create `specs.md`

```
# CityOps AI — workshop specification

## Goal
Classify synthetic municipal requests, propose a team, and produce an auditable plan.

## In scope
- Streetlight, water leak, and road damage examples.
- Deterministic classification and optional model suggestion.
- Human-review escalation for unknown or conflicting cases.
- Local JSON state, JSONL audit, and draft-only tool operations.
- Supervisor roles for triage, routing, and safety review.
- Automated tests and repeatable evaluation.

## Out of scope
- Real dispatch, emergency response, citizen identity, payments, or production work orders.
- Real municipal integrations or personal data.
- Treating an LLM response as policy or authorization.

## Acceptance criteria
1. Known synthetic cases map to the expected service team.
2. Unknown/empty descriptions escalate to Human Review.
3. Conflicting or unsupported classifications fail closed.
4. A plan never sets `can_auto_dispatch` to true.
5. Unit tests and evaluation run without an API key.
6. Generated state, audit, and secrets are not committed.
7. Every code change is reviewed through a pull request.

## Human decision
A real deployment would require identity-aware approval, authorization checks,
operational monitoring, data retention controls, and formal safety review.
This workshop is not a production deployment blueprint.
```

6.5 Run the full validation

```
python -m unittest -v
python evaluate.py
```

Expected: six unit tests pass and the evaluation reports 5/5. If a case fails, inspect the printed checks and update the rules/tests together.

Stage 7 — GitHub Actions and security controls

Concept

Use pull requests as the control point. CI runs tests; CodeQL checks source code; Dependabot raises dependency updates; dependency review checks proposed dependency changes. Availability of some security features depends on repository visibility and GitHub plan.

7.1 Create `.github/workflows/ci.yml`

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
      - name: Run unit tests
        run: python -m unittest -v
      - name: Run evaluation
        run: python evaluate.py
```

7.2 Create `.github/workflows/codeql.yml`

```
name: CodeQL
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: "23 5 * * 2"
  workflow_dispatch:

permissions:
  contents: read
  security-events: write
  actions: read

jobs:
```

```
  analyze:
    name: Analyze Python
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
      actions: read
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Initialize CodeQL
        uses: github/codeql-action/init@v4
        with:
          languages: python
          queries: security-extended
      - name: Autobuild
        uses: github/codeql-action/autobuild@v4
      - name: Analyze
        uses: github/codeql-action/analyze@v4
```

7.3 Create `.github/dependabot.yml`

```
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
```

7.4 Create `.github/workflows/dependency-review.yml`

```
name: Dependency Review
on:
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Review dependency changes
        uses: actions/dependency-review-action@v4
        with:
          fail-on-severity: high
```

7.5 Create `SECURITY.md`

```
# Security Policy

## Scope
This is a synthetic workshop demo. Do not submit real citizen data, secrets,
or production incidents to this repository.

## Reporting
For a workshop issue, open a private discussion with the workshop owner or
use the repository's private vulnerability reporting feature if enabled.
Do not publish credentials or sensitive details in a public issue.

## Demo limitations
- The app does not dispatch crews or call municipal systems.
- Local audit files are not tamper-proof evidence.
- The approver label is not authentication.
- A real deployment requires security, privacy, safety, and operational review.
```

7.6 Create `.github/pull_request_template.md`

```
## Change summary
Describe the change and why it is needed.

## Validation
- [ ] `python -m unittest -v`
- [ ] `python evaluate.py`
- [ ] No real citizen data or secrets included
- [ ] Unknown/conflicting requests still escalate
- [ ] No code path can dispatch real work
- [ ] Documentation updated

## Risk and rollback
Describe possible impact and how to revert the change.
```

7.7 Push and inspect

```
git add .
git commit -m "stage 7: add CI and security workflows"
git push
```

• Open the repository’s Actions tab and inspect the CI run.

• Create a branch and pull request; confirm tests run for the pull request.

• Review CodeQL results in the Security tab when the feature is available.

• Review Dependabot alerts and dependency update pull requests.

• Never add an API key to a workflow file. If a future CI step needs a key, use a GitHub Actions secret and only for trusted workflows.

Optional SBOM/SARIF learning exercise

Discuss what an SBOM represents and why SARIF standardizes static-analysis findings. This minimal repo does not need a custom SBOM generator or manual SARIF upload to complete the core workshop; use the repository’s available dependency/security features and inspect exported artifacts only when the feature is available.

Stage 8 — Copilot workflow, governance, and capstone

Concept

Use Copilot as a development assistant under repository guidance, not as an unrestricted production operator. Plan work in Issues, use a branch and PR, run checks, inspect the diff, and record a human decision.

8.1 Replace `README.md`

````
# CityOps AI

A small GH-600 workshop repository for synthetic municipal service triage.

## Safety
This is a teaching simulation. It does not dispatch crews, contact emergency
services, or create real municipal work orders. Use synthetic data only.

## Requirements
- Python 3.11+
- Optional OpenAI API key for a suggestion step
- Optional compatible MCP client for local tools

## Setup (PowerShell)
```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
```

Put an OpenAI key in `.env` only if you want to test the optional model step.
Never commit `.env`.

## Run
```powershell
python -m unittest -v
python evaluate.py
python app.py plan --description "Streetlight is not working"
python app.py create --id DEMO-001 --description "Streetlight is not working"
python app.py show --id DEMO-001
python app.py approve --id DEMO-001 --approver "Workshop Facilitator"
```

The approval command is demo state only; it is not real authentication.

## Architecture
- Deterministic classification: policy baseline.
- OpenAI suggestion: optional and fallible.
- Tools/MCP: synthetic, bounded functions.
- State/audit: local JSON and JSONL.
- Supervisor: triage, routing, safety review.
- Evaluation: unittest plus repeatable `cases.json`.
- GitHub: pull requests, CI, CodeQL, Dependabot, dependency review.

## Change workflow
1. Open an Issue with expected behaviour and acceptance criteria.
2. Create a short-lived feature branch.
3. Ask Copilot to plan; review the plan before code changes.
````

```
4. Implement one small change and add tests.
5. Run `python -m unittest -v` and `python evaluate.py`.
6. Open a PR and review the full diff.
7. Merge only when required checks pass and a human approves.
```

8.2 Create `.github/prompts/investigate.prompt.md`

```
---
mode: agent
description: Investigate a CityOps behaviour safely before changing code
---

Investigate this CityOps AI issue before editing files.

1. Read `.github/copilot-instructions.md`, `specs.md`, and relevant tests.
2. Summarize the current behaviour and identify the smallest likely cause.
3. Propose a plan with files to inspect and tests to run.
4. Do not edit files until the user approves the plan.
5. Preserve deterministic validation and human-review escalation.
6. Do not add real dispatch, external integrations, or personal data.
7. After approval, make the smallest change and add a regression test.
8. Run `python -m unittest -v` and `python evaluate.py`.
9. Report changed files, test results, remaining risks, and rollback steps.
```

8.3 Create `.github/agents/safety-reviewer.agent.md`

```
---
name: safety-reviewer
description: Review CityOps changes for unsafe automation and policy bypasses
---

You are a reviewer, not a dispatcher. Inspect the proposed diff and report:
- Whether unknown or conflicting inputs still escalate to Human Review.
- Whether deterministic validation remains authoritative.
- Whether any path can auto-dispatch or call a real external service.
- Whether secrets or personal data could be exposed.
- Whether tests cover normal, unknown, malformed, and conflicting inputs.
- Whether audit claims are accurate and do not imply tamper-proof storage.
Return findings as Critical, High, Medium, or Low with file/line references.
Do not modify files and do not approve a change with a Critical or High finding.
```

8.4 Create `.github/skills/cityops-review/SKILL.md`

```
---
name: cityops-review
description: Repeatable checklist for reviewing CityOps AI changes
---

# CityOps review skill

When asked to review a change:
1. Read the repository instructions and `specs.md`.
2. Identify the user-visible behaviour change.
3. Check deterministic rules, input bounds, and fail-closed handling.
4. Check human review and confirm `can_auto_dispatch` remains false.
5. Inspect tests and evaluation cases for regression coverage.
6. Check that `.env`, local state, audit output, and personal data are not committed.
7. Run `python -m unittest -v` and `python evaluate.py` when execution is available.
8. Report findings first, then tests, then residual risks.
Never invoke real municipal services or treat a model output as authorization.
```

8.5 GitHub governance checklist

• Protect `main` with a pull request requirement.

• Require the CI status check before merge where supported.

• Require at least one reviewer for workshop changes.

• Restrict force pushes and branch deletion on `main` where supported.

• Enable secret scanning and push protection if available for the repository/plan.

• Review CodeQL and Dependabot alerts; do not dismiss alerts without a reason.

• Use GitHub Actions permissions minimally; workflows should not receive write access unless required.

8.6 Capstone exercise

Create an Issue: “Add a safe park-maintenance request category.” Define acceptance criteria: new category maps to a known team, unknown requests still escalate, no auto-dispatch, tests and evaluation pass, docs updated.

• Create branch `feature/park-maintenance`.

• Use Copilot to plan and ask questions; review the plan before implementation.

• Implement the change using complete-file edits and add regression tests/cases.

• Run tests and evaluation locally.

• Open a pull request using the template; review the diff and workflow results.

• Ask the safety-reviewer agent to inspect the diff and address any findings.

• Merge only after a human review and required checks pass.

8.7 Final validation commands

```
python -m unittest -v
python evaluate.py
git status
git diff --check
git diff
```

Final success criteria: tests and evaluation pass; the key remains out of Git; unknown/conflicting cases require human review; no automatic dispatch exists; a pull request documents the change and has review evidence.

Troubleshooting

Python or virtual environment

• If `python` is not found, install Python and reopen PowerShell; verify with `python --version`.

• If activation is blocked, use `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` in the current PowerShell session, then activate again.

• If imports fail, activate `.venv` and run `python -m pip install -r requirements.txt`.

OpenAI step

• If the key is absent, the suggestion function returns `None` and deterministic rules continue.

• If the API call fails, inspect connectivity, account/model access, and local environment variables; the demo intentionally falls back.

• Do not paste API keys into source files or commit history.

MCP step

• An MCP server using stdio can appear to wait with no console output; it is waiting for a client.

• Use the exact Python interpreter/absolute path required by the MCP client configuration.

• Client configuration formats differ; validate the config against the client's current documentation.

GitHub Actions/security

• If a workflow is not triggered, confirm the file is under `.github/workflows/` and the branch/event matches the YAML.

• Security features can vary by plan, repository visibility, and organization policy.

• A workflow status check must be selected by its exact displayed name in branch protection/ruleset settings.

Final repository tree

```
cityops-ai/
■■■ app.py
■■■ openai_client.py
■■■ tools.py
■■■ mcp_server.py
■■■ state.py
■■■ cases.json
■■■ evaluate.py
■■■ test_app.py
■■■ specs.md
■■■ requirements.txt
■■■ .env.example
■■■ .gitignore
■■■ README.md
■■■ .github/
```

```
    ■■■ copilot-instructions.md
    ■■■ prompts/
    ■   ■■■ investigate.prompt.md
    ■■■ agents/
    ■   ■■■ safety-reviewer.agent.md
    ■■■ skills/
    ■   ■■■ cityops-review/
    ■       ■■■ SKILL.md
    ■■■ workflows/
    ■   ■■■ ci.yml
    ■   ■■■ codeql.yml
    ■   ■■■ dependency-review.yml
    ■■■ dependabot.yml
    ■■■ pull_request_template.md
SECURITY.md
```

Note: The tree lists every workshop file. `cityops_state.json` and `cityops_audit.jsonl` are generated at runtime and should remain untracked.